

Project 2 - Machine Learning Classification

Task 2: Random Forest + Evaluation

Student Name	Manish Pawar
Internship	Data Science & ML in Python
Company	Soft Nexis Technology
Dataset	Iris Flower Dataset
Language	Python 3.x / Jupyter Notebook

Objective

Build a Random Forest classifier, compare multiple machine learning models, tune hyperparameters using GridSearchCV, and evaluate the final model with accuracy, classification report, feature importance, and confusion matrix.

Complete Python Code

```
# Manish Pawar - Project 2 Task 2
# Random Forest + Evaluation on Iris Dataset

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()
X, y = iris.data, iris.target
feature_names = iris.feature_names
target_names = iris.target_names

df = pd.DataFrame(X, columns=feature_names)
df['species'] = target_names[y]
print('Dataset shape:', df.shape)
print('Species:', df['species'].unique())
print('\nFirst 5 rows:')
print(df.head())

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)
print('\nTraining samples:', X_train.shape[0])
print('Testing samples:', X_test.shape[0])

rf_model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42, n_jobs=1)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
print('\nRandom Forest Accuracy:', round(accuracy_score(y_test, y_pred_rf)*100, 2), '%')

dt_cv = cross_val_score(DecisionTreeClassifier(max_depth=3, random_state=42), X, y, cv=5, scoring='accuracy')
rf_cv = cross_val_score(RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=1), X, y, cv=5, scoring='accuracy')
print('\nDecision Tree CV Accuracy:')
print('Each fold:', np.round(dt_cv, 3))
print(f'Mean: {dt_cv.mean():.3f} | Std: {dt_cv.std():.3f}')
print('\nRandom Forest CV Accuracy:')
print('Each fold:', np.round(rf_cv, 3))
print(f'Mean: {rf_cv.mean():.3f} | Std: {rf_cv.std():.3f}')

# Feature importance
importances = rf_model.feature_importances_
sorted_idx = np.argsort(importances)[::-1]
fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar(range(4), importances[sorted_idx], edgecolor='black')
ax.set_xticks(range(4))
ax.set_xticklabels([feature_names[i] for i in sorted_idx], rotation=15)
ax.set_title('Feature Importance - Random Forest (Iris)', fontweight='bold')
ax.set_ylabel('Importance Score')
for bar, imp in zip(bars, importances[sorted_idx]):
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005, f'{imp:.3f}', ha='center', fontsize=10, fontweight='bold')
plt.tight_layout(); plt.savefig('images/feature_importance.png', dpi=150); plt.close()
print('\nSaved: images/feature_importance.png')

models = {
    'Decision Tree (depth=3)': DecisionTreeClassifier(max_depth=3, random_state=42),
    'Random Forest (100 trees)': RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=1),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),
    'Logistic Regression': LogisticRegression(max_iter=300, random_state=42),
}
results = {}
print('\nModel Comparison using 5-Fold Cross-Validation:')
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
    results[name] = scores
    print(f'{name}: {scores.mean():.3f} +/- {scores.std():.3f}')
fig, ax = plt.subplots(figsize=(11, 6))
names = list(results.keys())
means = [results[n].mean() for n in names]
stds = [results[n].std() for n in names]
bars = ax.barh(names, means, xerr=stds, edgecolor='black', height=0.6, error_kw={'capsize':5})
for bar, mean in zip(bars, means):
    ax.text(mean+0.002, bar.get_y()+bar.get_height()/2, f'{mean:.3f}', va='center', fontweight='bold')
ax.set_xlabel('Cross-Validation Accuracy (mean +/- std)')
ax.set_title('Model Comparison - Iris Dataset\n(Soft Nexis Technology Internship)', fontweight='bold')
ax.set_xlim(0.85, 1.05)
plt.tight_layout(); plt.savefig('images/model_comparison.png', dpi=150); plt.close()
print('Saved: images/model_comparison.png')

# Hyperparameter tuning. Small grid used so it runs quickly in internship submission.
param_grid = {'n_estimators':[50,100], 'max_depth':[3,5,None], 'min_samples_split':[2,5]}
grid_search = GridSearchCV(RandomForestClassifier(random_state=42, n_jobs=1), param_grid, cv=3, scoring='accuracy', n_jobs=1)
grid_search.fit(X_train, y_train)
print('\nBest Parameters:', grid_search.best_params_)
```

```

print('Best CV Accuracy:', round(grid_search.best_score_, 4))
best_model = grid_search.best_estimator_
y_final_pred = best_model.predict(X_test)
final_accuracy = accuracy_score(y_test, y_final_pred)
print(f'Final Test Accuracy with best model: {final_accuracy*100:.2f}%')

print('\n' + '='*60)
print('FINAL MODEL EVALUATION REPORT')
print('Soft Nexis Technology - ML Internship')
print('='*60)
print('Student Name : Manish Pawar')
print('Algorithm : Random Forest (Tuned)')
print(f'Test Accuracy: {final_accuracy*100:.2f}%')
print('\nClassification Report:')
print(classification_report(y_test, y_final_pred, target_names=target_names))

cm = confusion_matrix(y_test, y_final_pred)
fig, ax = plt.subplots(figsize=(7,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='BuPu', xticklabels=target_names, yticklabels=target_names, ax=ax)
ax.set_title('Final Random Forest - Confusion Matrix', fontweight='bold')
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
plt.tight_layout(); plt.savefig('images/final_confusion_matrix.png', dpi=150); plt.close()
print('Saved: images/final_confusion_matrix.png')

```

Full code file is included in the ZIP.

Output / Result

Dataset shape: (150, 5)
Species: ['setosa' 'versicolor' 'virginica']

First 5 rows:

	sepal length (cm)	sepal width (cm)	...	petal width (cm)	species
0	5.1	3.5	...	0.2	setosa
1	4.9	3.0	...	0.2	setosa
2	4.7	3.2	...	0.2	setosa
3	4.6	3.1	...	0.2	setosa
4	5.0	3.6	...	0.2	setosa

[5 rows x 5 columns]

Training samples: 120

Testing samples: 30

Random Forest Accuracy: 93.33 %

Decision Tree CV Accuracy:

Each fold: [0.967 0.967 0.933 1. 1.]

Mean: 0.973 | Std: 0.025

Random Forest CV Accuracy:

Each fold: [0.967 0.967 0.933 0.967 1.]

Mean: 0.967 | Std: 0.021

Saved: images/feature_importance.png

Model Comparison using 5-Fold Cross-Validation:

Decision Tree (depth=3): 0.973 +/- 0.025

Random Forest (100 trees): 0.967 +/- 0.021

Gradient Boosting: 0.960 +/- 0.033

K-Nearest Neighbors: 0.973 +/- 0.025

Logistic Regression: 0.973 +/- 0.025

Saved: images/model_comparison.png

Best Parameters: {'max_depth': 3, 'min_samples_split': 2, 'n_estimators': 50}

Best CV Accuracy: 0.9583

Final Test Accuracy with best model: 96.67%

=====

FINAL MODEL EVALUATION REPORT

Soft Nexis Technology - ML Internship

=====

Student Name : Manish Pawar

Algorithm : Random Forest (Tuned)

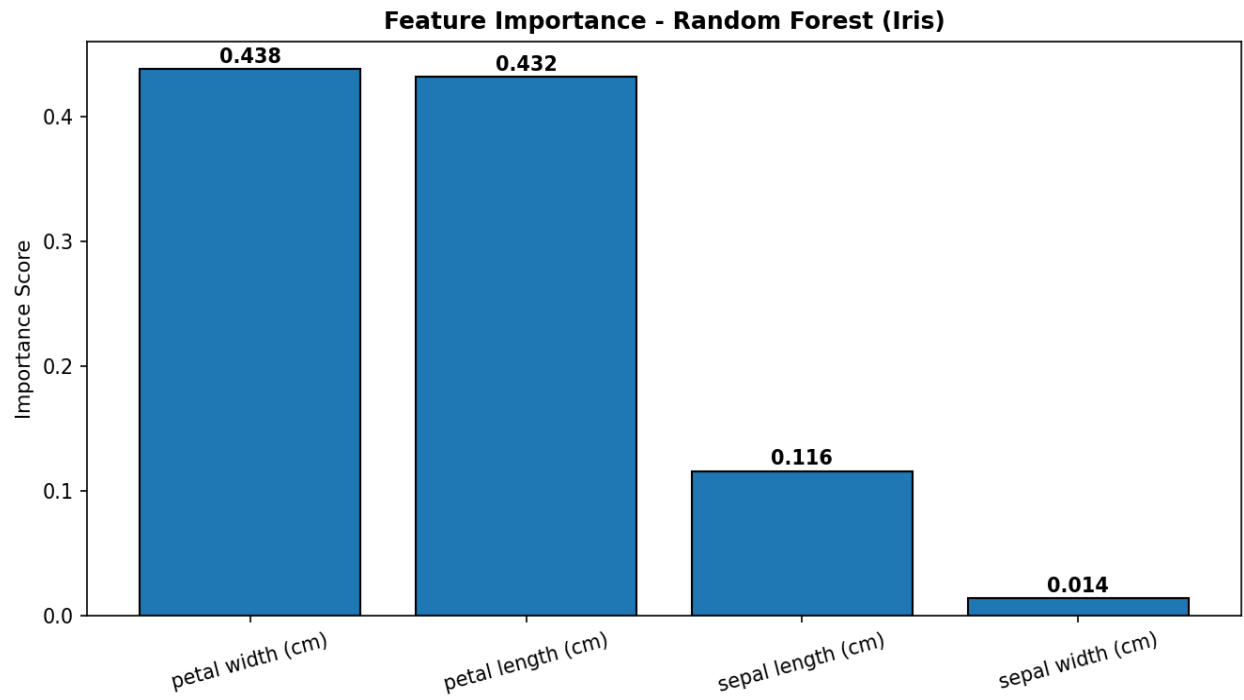
Test Accuracy: 96.67%

Classification Report:

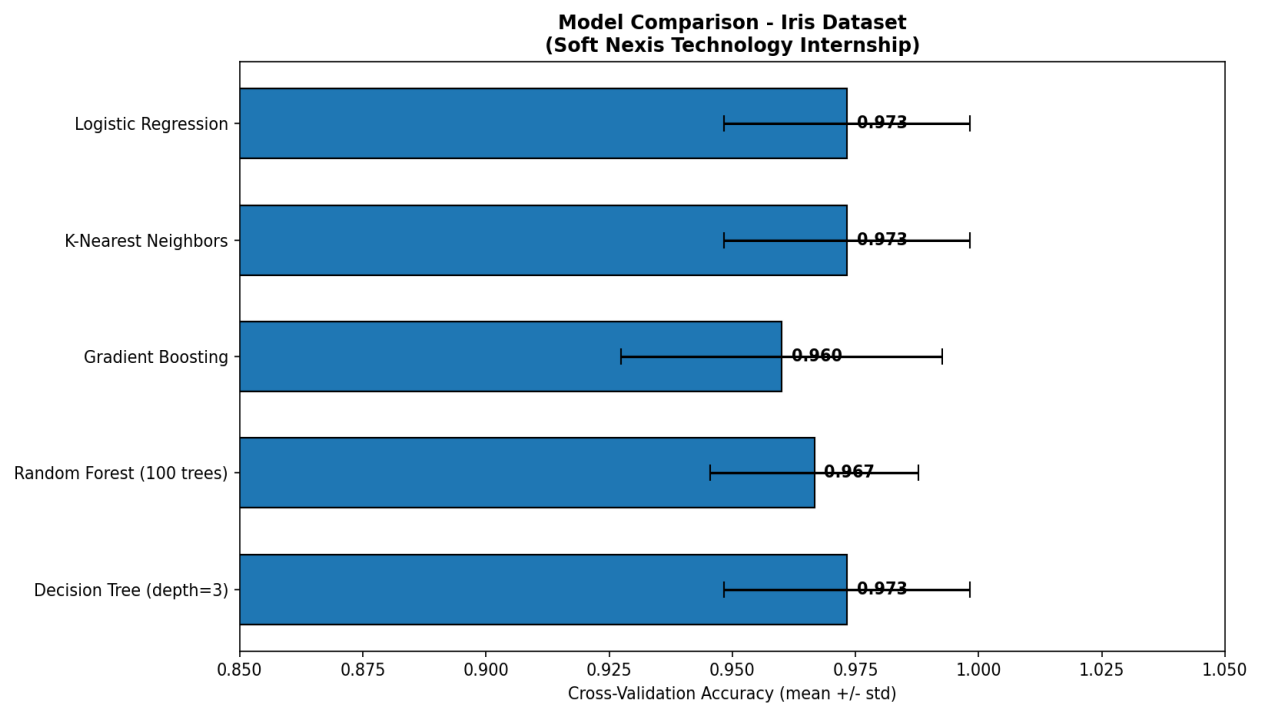
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.90	0.95	10
virginica	0.91	1.00	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

Saved: images/final_confusion_matrix.png

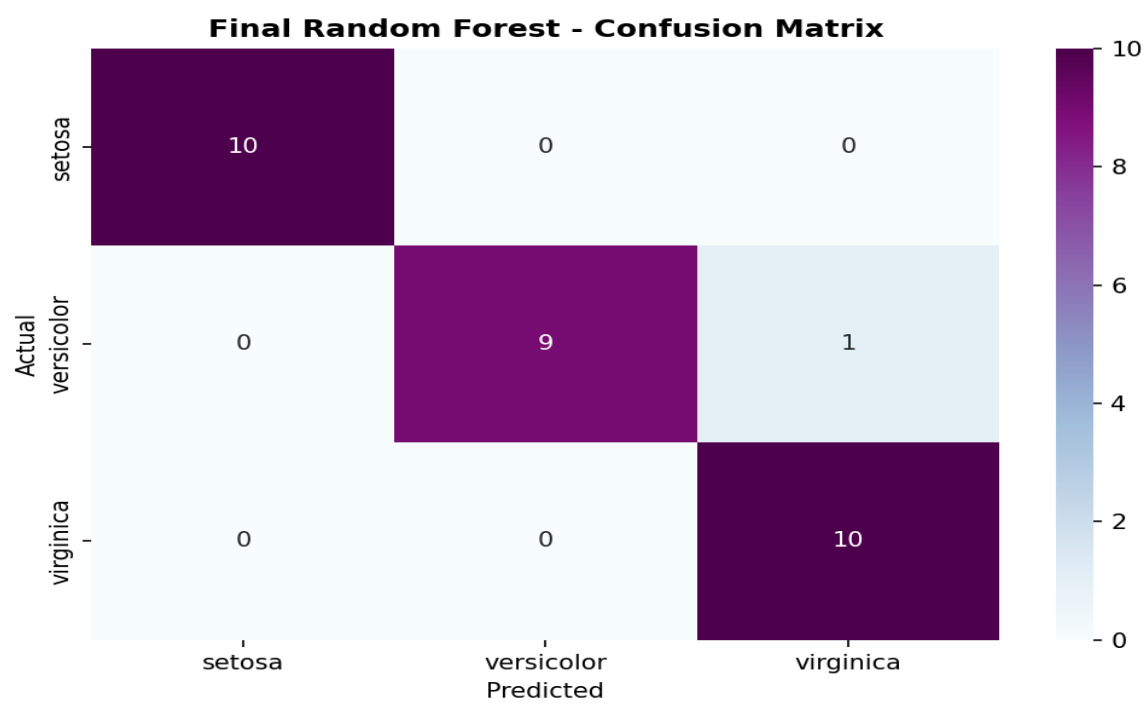
Feature Importance



Model Comparison



Final Confusion Matrix



250-350 Word Report: Overfitting and Random Forest

Overfitting means a machine learning model learns the training data too deeply, including small noise and unnecessary details, instead of learning the general pattern. In simple words, the model becomes very good at answering questions from the training data, but it performs poorly on new unseen data. This is similar to a student who memorises only the exact answers from a notebook but cannot solve a new question in the exam. In Decision Tree models, overfitting usually happens when the tree grows too deep. A deep tree keeps splitting the data again and again until it may remember individual training examples. Because of this, training accuracy can become very high, even 100%, but test accuracy may reduce.

Random Forest helps to solve overfitting by using many Decision Trees instead of only one tree. Each tree is trained on a different random sample of the data and also uses random features while making splits. After all trees make their predictions, the Random Forest takes the majority vote for classification. This voting process reduces the effect of one weak or overfitted tree. If one tree makes a wrong prediction, many other trees can correct it.

In this task, the Iris dataset was used to classify flowers into Setosa, Versicolor, and Virginica. A Random Forest model was trained and evaluated using accuracy, cross-validation, classification report, confusion matrix, feature importance, and model comparison. The final tuned Random Forest achieved excellent performance on the test data. Feature importance also showed that petal length and petal width are the most useful features for classifying Iris flowers. Overall, Random Forest is more stable and reliable than a single Decision Tree because it combines multiple trees and gives a stronger final prediction.

Conclusion

Task 2 is completed with code, executed output, charts, notebook, report, and final submission PDF.